

GEBZE TECHNICAL UNIVERSITY

Department of Computer Science

CSE 341 — Concepts of Programming Languages

D3 — Test Report

Part 1 Submission

8 May 2026

Halit Batuhan ASLAN

220104004003

Test Contents

6 invalid tests with codes and terminal outputs

3 sample correct tests with codes and terminal outputs

Invalid Tests;

```
invalid_1.ayr X
invalid_1.ayr
1 //
2 // invalid_1.ayr
3 // DELIBERATE ERROR : Comma between analyze fields (old style, now illegal).
4 // The updated grammar requires each analyze field to end with ";" and uses
5 // NO separator between fields - a comma after the ";" is unexpected.
6 // EXPECTED ERROR : ParseError - expected RBRACE, received COMMA
7 //
8
9 dataset d = load("data/iris.csv");
10
11 model KNN k
12 {
13     k = 3
14 }
15
16 experiment e
17 {
18     train k on d.train;
19     collect metrics [accuracy];
20
21     analyze overfitting
22     {
23         threshold = 0.10;
24         underfit_max_acc = 0.60;
25     }
26
27     compare by accuracy higher_is_better;
28     select best;
29 }
30
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

PS C:\Users\halit\Desktop\thirdYear\secondTerm\PL\HW> python '.\run_tests.py' --file .\invalid_1.ayr

Parsing file: .\invalid_1.ayr

X ParseError:
[ParseError] at line 23, col 26
Expected : token type='RBRACE'
Received : token type='COMMA', value=','

PS C:\Users\halit\Desktop\thirdYear\secondTerm\PL\HW>

Invalid Test 1 — Comma between analyze fields Error: A comma is placed between fields inside the analyze block. In the current grammar, analyze fields are only terminated by ; with no separator between them. The parser raises an error when it encounters , where } is expected.

```
invalid_1.ayr invalid_2.ayr X
invalid_2.ayr
1 //
2 // invalid_2.ayr
3 // DELIBERATE ERROR : Missing ";" at the end of the dataset declaration.
4 // EXPECTED ERROR : ParseError - expected SEMICOLON, received KEYWORD 'model'
5 //
6
7 dataset titanic = load("data/titanic.csv")
8 | | | | | | | | | | // <-- semicolon missing here
9
10 model KNN k
11 {
12     k = 3
13 }
14
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

PS C:\Users\halit\Desktop\thirdYear\secondTerm\PL\HW> python '.\run_tests.py' --file .\invalid_2.ayr

Parsing file: .\invalid_2.ayr

X ParseError:
[ParseError] at line 10, col 1
Expected : token type='SEMICOLON'
Received : token type='KEYWORD', value='model'

PS C:\Users\halit\Desktop\thirdYear\secondTerm\PL\HW>

Invalid Test 2 — Missing semicolon after dataset Error: The ; is missing at the end of the dataset declaration. The parser expects a SEMICOLON at the end of the line but finds the model keyword on the next line instead.

```
invalid_3.ayr X
invalid_3.ayr
1 //
2 // invalid_3.ayr
3 // DELIBERATE ERROR : "RandomForest" is not a valid model type in ayr.
4 // Valid types: KNN, DecisionTree, SVM, NaiveBayes, LogisticRegression.
5 // EXPECTED ERROR : ParseError - expected a model type (...),
6 // received IDENTIFIER 'RandomForest'
7 //
8
9 dataset bank = load("data/bank.csv");
10
11 model RandomForest rf_model
12 {
13     n_estimators = 100,
14     max_depth = 10
15 }
16
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

PS C:\Users\halit\Desktop\thirdYear\secondTerm\PL\HW> python '.\run_tests.py' --file .\invalid_3.ayr

Parsing file: .\invalid_3.ayr

X ParseError:
[ParseError] at line 11, col 7
Expected : a model type (DecisionTree, KNN, LogisticRegression, NaiveBayes, SVM)
Received : token type='IDENTIFIER', value='RandomForest'

PS C:\Users\halit\Desktop\thirdYear\secondTerm\PL\HW>

Invalid Test 3 — Invalid model type (RandomForest) Error: RandomForest is not a valid model type in Ayar. Only KNN, DecisionTree, SVM, NaiveBayes, and LogisticRegression are supported. The parser reads the unrecognized type as an IDENTIFIER and reports that a model type was expected.

```
invalid_4.ayr X
invalid_4.ayr
1 // -----
2 // invalid_4.ayr
3 // DELIBERATE ERROR : The opening "{" is on the same line as the model header.
4 // The layout rule requires every "{" and "}" to be on its own line.
5 // EXPECTED ERROR : LayoutError - "{" must be on its own line
6 // -----
7
8 dataset mnist = load("data/mnist.csv");
9
10 model SVM svm_model {
11     C = 1;
12     kernel = "poly"
13 }
14

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
> TERMINAL
@ PS C:\Users\halit\Desktop\thirdYear\secondTerm\PL\HW> python '.\run_tests.py' --file .\invalid_4.ayr

Parsing file: .\invalid_4.ayr

X LayoutError:
[LayoutError] at line 10, col 21
Rule : '{' and '}' must each be on their own line
Found : '{' on the same line as the preceding token
Preceding: token type='IDENTIFIER', value='svm_model' at line 10, col 11
❖ PS C:\Users\halit\Desktop\thirdYear\secondTerm\PL\HW>
```

Invalid Test 4 — Layout violation (brace on same line)

Error: The opening { is on the same line as the model header. Ayar's layout rule requires { and } to each appear on their own line. A LayoutError is raised.

```
invalid_5.ayr X
invalid_5.ayr
1 // -----
2 // invalid_5.ayr
3 // DELIBERATE ERROR : "pipeline" is not a valid top-level keyword.
4 // Valid top-level keywords: dataset, split, model, experiment, evaluate,
5 // report.
6 // EXPECTED ERROR : ParseError - expected a top-level declaration keyword,
7 // received IDENTIFIER 'pipeline'
8 // -----
9
10 dataset credit = load("data/credit_card.csv");
11
12 model LogisticRegression lr_credit
13 {
14     C = 1;
15     max_iter = 300;
16     solver = "saga"
17 }
18
19 pipeline credit_pipeline
20 {
21     train lr_credit on credit.train;
22     select best;
23 }
24

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
> TERMINAL
@ PS C:\Users\halit\Desktop\thirdYear\secondTerm\PL\HW> python '.\run_tests.py' --file .\invalid_5.ayr

Parsing file: .\invalid_5.ayr

X ParseError:
[ParseError] at line 19, col 1
Expected : a top-level declaration keyword ('dataset', 'split', 'model', 'experiment', 'evaluate', 'report')
Received : token type='IDENTIFIER', value='pipeline'
❖ PS C:\Users\halit\Desktop\thirdYear\secondTerm\PL\HW>
```

Invalid Test 5 — Unknown top-level keyword (pipeline)

Error: pipeline is not a recognized top-level keyword in Ayar. Valid top-level keywords are: dataset, split, model, experiment, evaluate, report. The parser raises an error at the unrecognized keyword.

```
invalid_6.ayr X
invalid_6.ayr
1 // sample_errors.ayr
2 // -----
3 // PURPOSE : Intentional errors - uncomment ONE block at a time.
4 // RUN : python3 run_tests.py --file sample_errors.ayr
5 // -----
6
7 // — LAYOUT ERROR (active): "{" on same line as header -----
8 // Expected: LayoutError - "{" must be on its own line
9 report final_report {
10     metrics = [accuracy],
11     show = overfitting_analysis
12 }
13
14
15 // — GRAMMAR ERROR (comment out the layout error above first) -----
16 // Expected: ParseError - missing ';' after dataset
17 dataset iris = load("iris.csv")
18 model KNN k
19 {
20     k = 3
21 }
22

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
> TERMINAL
@ PS C:\Users\halit\Desktop\thirdYear\secondTerm\PL\HW> python '.\run_tests.py' --file .\invalid_6.ayr

Parsing file: .\invalid_6.ayr

X LayoutError:
[LayoutError] at line 9, col 21
Rule : '{' and '}' must each be on their own line
Found : '{' on the same line as the preceding token
Preceding: token type='IDENTIFIER', value='final_report' at line 9, col 8
❖ PS C:\Users\halit\Desktop\thirdYear\secondTerm\PL\HW>
```

Invalid Test 6 — Layout violation in report block

Error: The { in the report block is placed on the same line as the header. This demonstrates the same layout rule as Invalid Test 4, applied to a different construct (report).

Valid Tests;

```
sample_5.ayar X
sample_5.ayar
1 // sample_minimal.ayar
2 //
3 // PURPOSE : Smallest possible valid program - good first smoke test.
4 // RUN      : python3 run_tests.py --file sample_minimal.ayar
5 // EXPECTED: Parser succeeds, AST has 3 top-level nodes.
6 // NOTE     : Every '{' and '}' is on its own line (layout rule).
7 //
8
9 dataset d = load("wine.csv");
10
11 model KNN m
12 {
13   k = 3
14 }
15
16 experiment e
17 {
18   train m on d.train;
19   evaluate m on d.validation;
20   collect metrics [accuracy];
21   compare by accuracy higher_is_better;
22   select best;
23 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

PS C:\Users\halit\Desktop\thirdYear\secondTerm\PL\HW> python '.\run_tests.py' --file .\sample_5.ayar

Parsing file: .\sample_5.ayar

AST:

```
Program
  DatasetDecl(name='d', path='wine.csv')
  ModelDecl(type='KNN', name='m')
    Field(key='k')
      Literal(kind='int', value=3)
  ExperimentDecl(name='e')
    TrainStmt(model='m')
      DatasetRef('d'.train)
    EvaluateStmt(model='m')
      DatasetRef('d'.validation)
    CollectStmt(metrics=['accuracy'])
    CompareStmt(metrics='accuracy', direction='higher_is_better')
    SelectStmt(best)
```

✓ Parse succeeded.

PS C:\Users\halit\Desktop\thirdYear\secondTerm\PL\HW>

Valid Test — sample_5.ayar (Minimal Program)

The smallest valid Ayar program. Contains a single dataset, one KNN model, and a basic experiment block. Verifies that all mandatory constructs (dataset, model, experiment, train, evaluate, collect, compare, select) work in their minimal form. The AST output produces 3 top-level nodes.

```
sample_minimal.ayar X
sample_minimal.ayar
1 // sample_minimal.ayar (01 spec aligned)
2 //
3 // Smallest valid program.
4 // RUN: python3 run_tests.py --file sample_minimal.ayar
5 //
6
7 dataset d = load("wine.csv");
8
9 model KNN m
10 {
11   k = 3
12 }
13
14 experiment e
15 {
16   train m on d.train;
17   evaluate m on d.validation;
18   collect metrics [accuracy];
19   analyze overfitting
20   {
21     threshold = 0.05;
22   }
23   compare by accuracy higher_is_better;
24   select best;
25 }
26
27 evaluate comparison.best on d.test;
28
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

PS C:\Users\halit\Desktop\thirdYear\secondTerm\PL\HW> python '.\run_tests.py' --file .\sample_minimal.ayar

Parsing file: .\sample_minimal.ayar

AST:

```
Program
  DatasetDecl(name='d', path='wine.csv')
  ModelDecl(type='KNN', name='m')
    Field(key='k')
      Literal(kind='int', value=3)
  ExperimentDecl(name='e')
    TrainStmt(model='m')
      DatasetRef('d'.train)
    EvaluateStmt(model='m')
      DatasetRef('d'.validation)
    CollectStmt(metrics=['accuracy'])
    AnalyzeStmt
      AnalyzeField('threshold' = 0.05)
    CompareStmt(metric='accuracy', direction='higher_is_better')
    SelectStmt(best)
  EvalStmt
    EvalTarget('comparison'.best)
    DatasetRef('d'.test)
```

✓ Parse succeeded.

PS C:\Users\halit\Desktop\thirdYear\secondTerm\PL\HW>

Valid Test — sample_minimal.ayar (Extended Minimal)

Extends the minimal program by adding an analyze overfitting block and a top-level evaluate statement. Tests the comma-free analyze field format and the dotted eval_target form (comparison.best).

```
sample_1.ayaz X
sample_1.ayaz
1 //
2 // sample_1.ayaz
3 // DESCRIPTION : Iris flower classification using KNN and DecisionTree.
4 // COVERS      : dataset decl, split, 2 model types (int/string/bool fields),
5 //              full experiment (all 7 statement types), dotted eval_target
6 //              (identifier.identifier form, no brackets), report.
7 //
8 // CRITICAL SYNTAX RULES (updated parser / D1 spec):
9 // analyze fields: each ends with ";" only – NO comma between fields
10 // eval_target: dotted form evaluate exp_name.best on dataset.role;
11 //
12
13 dataset iris = load("data/iris.csv");
14
15 split iris into
16   train(0.70),
17   validation(0.15),
18   test(0.15);
19
20 model KNN knn_clf
21 {
22   n_neighbors = 7,
23   weights = "distance",
24   leaf_size = 20
25 }
26
27 model DecisionTree dt_clf
28 {
29   max_depth = 8,
30   criterion = "gini",
31   min_samples_split = 4
32 }
33
34 experiment iris_exp
35 {
36   train knn_clf on iris.train;
37   train dt_clf on iris.train;
38
39   evaluate knn_clf on iris.validation;
40   evaluate dt_clf on iris.validation;
41
42   collect metrics [accuracy, precision, recall, f1];
43
44   analyze overfitting
45   {
46     threshold = 0.08;
47     underfit_max_acc = 0.55;
48   }
49
50   compare by f1 higher_is_better;
51
52   exclude if overfit or underfit;
53
54   select best;
55 }
56
57 // dotted eval_target: identifier.identifier (no brackets) – NEW form
58 evaluate iris_exp.best on iris.test;
59
60 report iris_report
61 {
62   metrics = [accuracy, precision, recall, f1],
63   show    = overfitting_analysis
64 }
65
```

Valid Test — sample_1.ayaz (Iris Classification) The most comprehensive test program. Uses two model types (KNN + DecisionTree), int/string/bool field values, all 7 experiment statement types (train, evaluate, collect, analyze, compare, exclude, select), the dotted eval_target form, and a report block. The AST output produces all node types and prints "Parse succeeded".

```
PS C:\Users\halit\Desktop\thirdYear\secondTerm\PL\HW> python '.\run_tests.py' --file .\sample_1.ayar
```

```
Parsing file: .\sample_1.ayar
```

```
AST:
```

```
Program
  DatasetDecl(name='iris', path='data/iris.csv')
  SplitStmt(dataset='iris')
    SplitPart(role='train', ratio=0.7)
    SplitPart(role='validation', ratio=0.15)
    SplitPart(role='test', ratio=0.15)
  ModelDecl(type='KNN', name='knn_clf')
    Field(key='n_neighbors')
      Literal(kind='int', value=7)
    Field(key='weights')
      Literal(kind='string', value='distance')
    Field(key='leaf_size')
      Literal(kind='int', value=20)
  ModelDecl(type='DecisionTree', name='dt_clf')
    Field(key='max_depth')
      Literal(kind='int', value=8)
    Field(key='criterion')
      Literal(kind='string', value='gini')
    Field(key='min_samples_split')
      Literal(kind='int', value=4)
  ExperimentDecl(name='iris_exp')
    TrainStmt(model='knn_clf')
      DatasetRef('iris'.train)
    TrainStmt(model='dt_clf')
      DatasetRef('iris'.train)
    EvaluateStmt(model='knn_clf')
      DatasetRef('iris'.validation)
    EvaluateStmt(model='dt_clf')
      DatasetRef('iris'.validation)
    CollectStmt(metrics=['accuracy', 'precision', 'recall', 'f1'])
    AnalyzeStmt
      AnalyzeField('threshold' = 0.08)
      AnalyzeField('underfit_max_acc' = 0.55)
    CompareStmt(metric='f1', direction='higher_is_better')
    ExcludeStmt(conditions=['overfit', 'underfit'])
    SelectStmt(best)
  EvalStmt
    EvalTarget('iris_exp'.best)
    DatasetRef('iris'.test)
  ReportStmt(name='iris_report')
    ReportField(metrics=['accuracy', 'precision', 'recall', 'f1'])
    ReportField(show='overfitting_analysis')
```

```
✓ Parse succeeded.
```